```
"novel": true
```

Idea 13/50 - label_noise_grokking

```
"Name": "label_noise_grokking",
"Title": "Grokking Under Noise: The Impact of Systematic and Random Label
Errors on Algorithmic Learning",
"Experiment": "1. Modify the AbstractDataset class to introduce two types
of label noise: random (labels changed randomly) and systematic (specific
labels consistently flipped). Add a 'noise_type' parameter
(random/systematic) and 'noise_level' parameter (low: 5%
high: 20%
training set while keeping the validation set clean. 3. Run experiments for
each noise type and level across all datasets. 4. Track metrics: time to
grokking (99%
loss, and model confidence (mean softmax probability of correct class). 5.
Plot learning curves and model confidence for different noise types and
levels, highlighting the grokking point for each. 6. Analyze how different
types and levels of label noise affect the timing, speed, and extent of
grokking across different operations. 7. Compare attention patterns between
noisy and clean training at different stages to understand how the model
adapts to noise.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 14/50 - compositional_grokking

```
"Name": "compositional_grokking",
"Title": "Compositional Grokking: Investigating the Relationship Between
Grokking and Compositional Learning in Modular Arithmetic",
"Experiment": "1. Modify ModSumDataset and ModSubtractDataset to include
composite operations: (a + b) - c mod p and (a - b) + c mod p. 2. Implement
new dataset class CompositeModDataset for these operations. 3. Run
experiments comparing learning curves for basic (a + b, a - b) and
composite operations. 4. Track metrics: time to grokking for basic vs.
composite operations, correlation between grokking times, final accuracies.
5. Analyze attention patterns to see if the model learns to attend to
intermediate results in composite operations. 6. Implement a 'compositional
generalization' test by training on basic operations and testing on their
compositions. 7. Compare internal representations (e.g., using PCA on
hidden states) for basic vs. composite operations at different stages of
training.",
"Interestingness": 9,
"Feasibility": 6,
"Novelty": 9,
"novel": true
```

Idea 15/50 - mutual_information_grokking

```
"Name": "mutual_information_grokking",
"Title": "Information Dynamics in Grokking: Analyzing Mutual Information
```

```
Evolution During Algorithmic Learning",
"Experiment": "1. Implement a function to estimate mutual information using
a binning approach for efficiency. 2. Modify the Transformer class to
output hidden states from the final layer. 3. Update the training loop to
calculate and store mutual information between (a) inputs and outputs, and
(b) final hidden states and outputs, at regular intervals. 4. Run
experiments across all datasets, tracking these mutual information metrics
alongside validation accuracy and training loss. 5. Create plots showing
the evolution of both mutual information metrics over training time,
highlighting the grokking point. 6. Analyze how mutual information trends
relate to grokking by testing specific hypotheses: (a) Rapid increase in
hidden state-output mutual information coincides with grokking, (b) Input-
output mutual information stabilizes post-grokking. 7. Compare mutual
information dynamics between different operations and model sizes to
identify common patterns in successful grokking.",
"Interestingness": 9,
"Feasibility": 6,
"Novelty": 9,
"novel": true
```

Idea 16/50 - sparse_subnetworks_grokking

```
"Name": "sparse_subnetworks_grokking",
"Title": "Sparse Subnetworks in Grokking: Investigating the Emergence of
Critical Structures During Algorithmic Learning",
"Experiment": "1. Implement a simple magnitude-based pruning function for
the Transformer model. 2. Modify the training loop to perform pruning at
key points: before training, just before grokking (based on validation
accuracy), and after grokking. 3. For each pruning point, create sparse
networks at different sparsity levels (e.g., 50%
these sparse networks from the original initialization for a fixed number
of steps. 5. Track metrics: validation accuracy of sparse networks,
sparsity level, and grokking speed (if it occurs). 6. Plot the performance
of sparse networks at different sparsity levels and pruning points. 7.
Compare the structure and performance of sparse networks found before and
after grokking across different operations.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 17/50 - positional_encoding_grokking

```
"Name": "positional_encoding_grokking",
"Title": "Inductive Biases in Grokking: The Impact of Positional Encoding
Schemes on Algorithmic Learning",
"Experiment": "1. Modify the Transformer class to support three positional
encoding schemes: sinusoidal (current), learned embeddings, and a simple
binary encoding (e.g., [0,1,0,1,0] for 'a o b = c'). 2. Implement these
encoding schemes, ensuring they work with the existing sequence length. 3.
Run experiments for each encoding scheme across all datasets, tracking:
time to grokking (99%
training loss, and attention entropy. 4. Analyze how different encoding
schemes affect attention patterns and grokking behavior for each operation
```

```
type. 5. Compare generalization capabilities on sequences with shuffled
operands (e.g., 'b o a = c'). 6. Correlate encoding scheme performance with
operation complexity to identify potential interactions between input
representation and task structure. 7. Discuss implications for designing
transformers for specific algorithmic tasks based on findings.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```

Idea 18/50 - adversarial_robustness_grokking

```
"Name": "adversarial_robustness_grokking",
"Title": "Adversarial Robustness During Grokking: Tracking Vulnerability
Evolution in Algorithmic Learning",
"Experiment": "1. Implement a simple perturbation method: randomly flip 1-2
bits in the input representation for modular arithmetic, and swap 1-2
elements for permutations. 2. Modify the training loop to generate
perturbed inputs and evaluate model performance on them every 500 steps. 3.
Track metrics: normal validation accuracy, accuracy on perturbed inputs,
and 'robustness gap' (difference between normal and perturbed accuracy). 4.
Plot the evolution of robustness to perturbations alongside the grokking
curve. 5. Compare robustness before, during, and after grokking across
different operations. 6. Analyze examples of successful perturbations at
different stages of training. 7. Investigate potential correlations between
the timing of grokking and changes in robustness to perturbations.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```

Idea 19/50 - critical_periods_grokking

```
"Name": "critical_periods_grokking",
"Title": "Critical Periods in Grokking: The Impact of Timed Learning Rate
Spikes on Algorithmic Learning",
"Experiment": "1. Modify the training loop to support learning rate spikes
at specific points (25%
to apply these spikes, increasing the learning rate by 10x for 100 steps.
3. Run experiments for each spike timing across all datasets (modular
arithmetic and permutations), including a control group with no spikes. 4.
Track metrics: time to grokking, final validation accuracy, and 'spike
impact' (change in validation accuracy slope in 500 steps post-spike). 5.
Plot learning curves highlighting spike points and their impacts. 6.
Analyze how spike timing affects grokking across different operations,
comparing modular arithmetic tasks with permutations. 7. Compare attention
patterns immediately before and after impactful spikes. 8. Correlate spike
impact with the stage of learning (pre-grokking, during grokking, post-
grokking) to identify potential critical periods, assessing whether these
periods are task-specific or general across operations.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 9,
"novel": true
```